

GAN Dashboard

DATS 6303: Final Project

Jeongwon Yoo, Pratik Shetty, Josh Schechter

Generative AI models have been increasingly improving over the years. For our final project, we sought a way to explore some of the backbone generative AI models, specifically for image generation. We selected to explore different variations of the Generative Adversarial Network (GAN) Architecture. We started with a simple DCGAN architecture, then worked on a WGAN-GP implementation, and finally attempted to train a ProGAN model. Once training was complete, we put together an interactive dashboard using streamlit, to display our results and demonstrate our understanding of the architectures. Our evaluation metric was Frechet Inception Distance (FID), which we selected because it both tracks diversity of its own images, and also the distance from a real image.

My Contribution

My section of the project focused on the implementation, training, and explanation of the DCGAN model. Since each team member mainly handled one model, I worked on building the DCGAN baseline, tuning its training setup, and making sure it was clearly explained in the Streamlit dashboard. I also worked on the DCGAN architecture section in the UI, where I added explanations of GAN basics, transposed convolution, generator and discriminator design, and the role of the discriminator loss in standard GAN training.

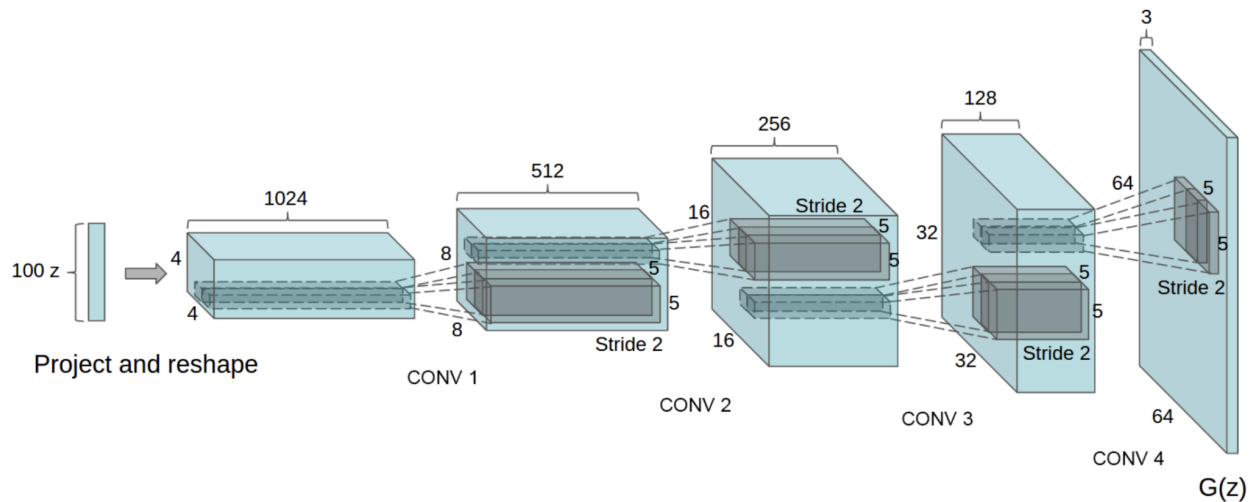
In addition, I helped with the live generation demo and some dashboard updates such as the overview, conclusions, and routing for the new page.

Architecture

DCGAN was our baseline GAN architecture and the first model we implemented. Like a standard GAN, it consists of a generator and a discriminator trained against each other.

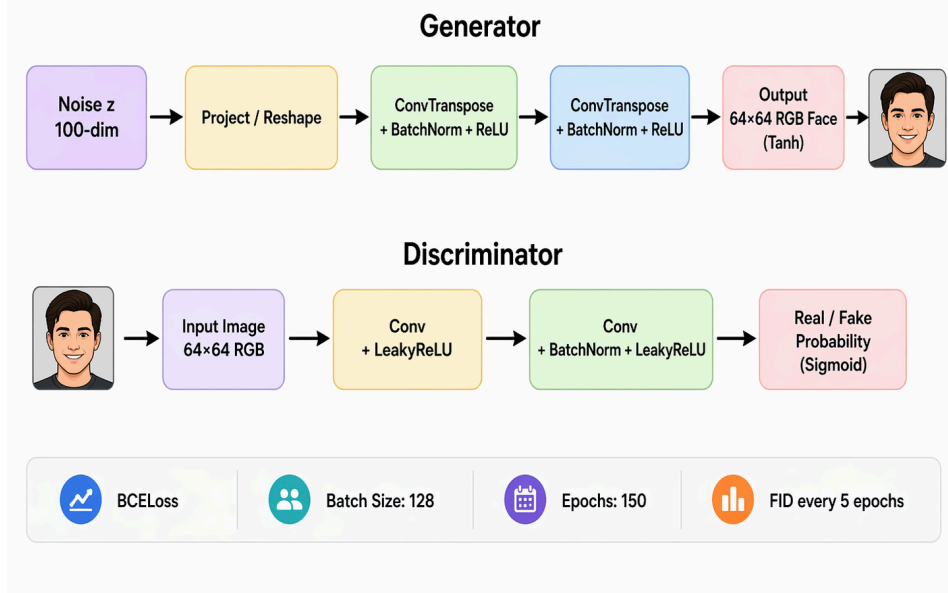
The generator takes a 100-dimensional latent noise vector and progressively

upsamples it into a 64×64 RGB face image using transposed convolution layers, Batch Normalization, and ReLU activations, followed by a final Tanh output layer.



The discriminator reverses this process by taking a 64×64 RGB image and downsampling it through convolution layers with LeakyReLU and Batch Normalization until it outputs a single real/fake probability. This convolutional design makes DCGAN much more appropriate for image generation than a simple fully connected GAN, and it served as a strong conceptual baseline before moving to WGAN-GP and ProGAN.

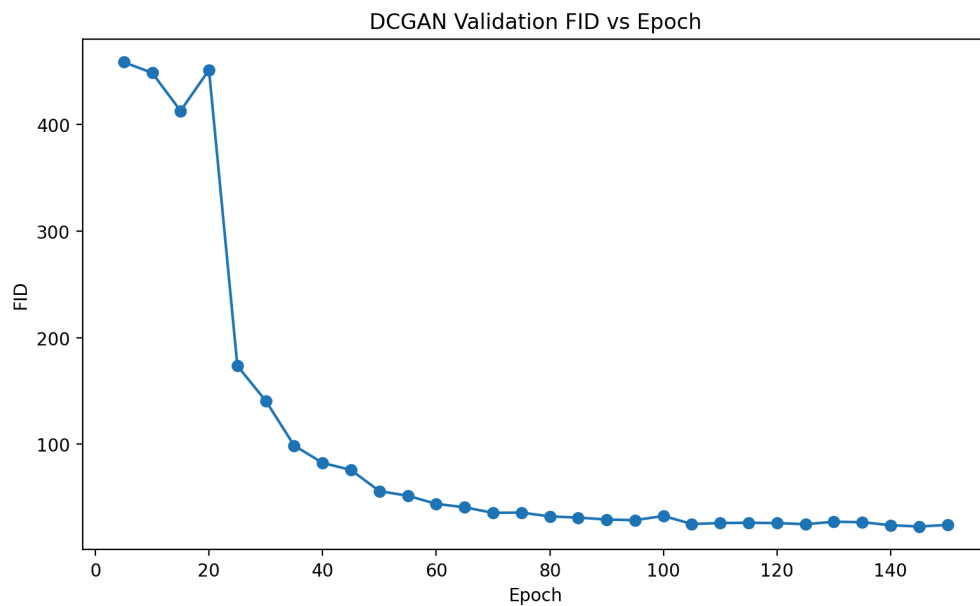
Simple DCGAN Architecture

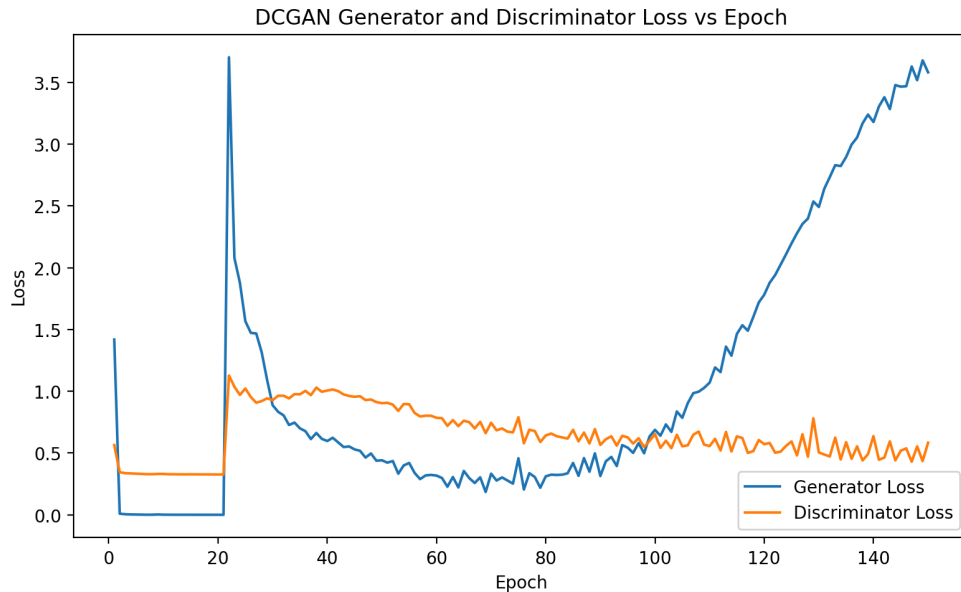


Training

The DCGAN model was trained only at 64×64 resolution and used as the baseline model in our comparison. We trained it with batch size 128, 150 epochs, and a feature-map width of 128. The training loop used binary cross-entropy loss, with separate Adam optimizers for the generator and discriminator. I set the generator learning rate to 1e-4 and the discriminator learning rate to 5e-5 so that the discriminator would not overpower the generator too early. To improve stability, I also added real-label smoothing at 0.9, annealed instance noise early in training, validation FID checks every five epochs, and checkpoint saving based on the best validation FID. After training, the best checkpoint was reloaded so the final model reflected the strongest validation performance rather than just the last epoch. Overall, DCGAN reached recognizable results fairly quickly, but it was still less stable than WGAN-GP, which is why it remained mainly a baseline and reference model in the final comparison.

The plots below summarize the DCGAN training behavior. The first plot shows validation FID over training epochs, where lower values indicate that the generated images are becoming closer to the real data distribution. The second plot shows generator and discriminator loss across epochs, which helps illustrate the adversarial learning behavior between the two networks during training. In our run, the validation FID improved substantially throughout training and eventually reached a best value of about 22.81.





Conclusions

Compared with the other models, DCGAN was not the strongest final performer, but it remained one of the most important parts of the project. It was the clearest model for understanding how adversarial image generation works, and it gave us a useful reference point for comparing later improvements. In practice, DCGAN trained faster to initial recognizable results than ProGAN, but it was more sensitive to instability than WGAN-GP. This helped show why later GAN variants were developed: DCGAN demonstrates the strengths of convolution-based image generation, but also the weaknesses of standard BCE-based GAN training when the generator and discriminator become unbalanced. For that reason, DCGAN was valuable both as a baseline model and as a conceptual foundation for understanding the rest of the project.

Code Percentages

None of my code was copied directly from an outside source. However, small helper logic and minor structural suggestions were sometimes developed with the assistance

of AI, especially for tasks such as UI wiring, helper utilities, or code cleanup. The main DCGAN architecture, training logic, and project-specific modifications were implemented and adapted directly for this codebase. If required, I can calculate my final assisted-code percentage separately based on the files I contributed to.

Results

The final DCGAN run produced recognizable 64×64 face images and improved steadily during training. Its best validation FID was about **22.81**, showing that the generated images became much closer to the real image distribution over time. Visually, DCGAN captured the basic structure of faces, but its outputs were still less stable and less refined than WGAN-GP. Overall, DCGAN worked well as a baseline and gave us a meaningful reference point for comparing the stronger models in the project.



DCGAN Image Example (64 pixels)

References

- Goodfellow et al. (2014), *Generative Adversarial Nets*
<https://arxiv.org/pdf/1406.2661>

- Radford et al. (2015), *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*
<https://arxiv.org/pdf/1511.06434>
- Arjovsky et al. (2017), *Wasserstein GAN*
<https://arxiv.org/abs/1701.07875>
- Gulrajani et al. (2017), *Improved Training of Wasserstein GANs*
<https://arxiv.org/abs/1704.00028>
- Karras et al. (2017), *Progressive Growing of GANs for Improved Quality, Stability, and Variation*
<https://arxiv.org/abs/1710.10196>
- Zeiler and Fergus (2010), *Deconvolutional Networks*
<https://www.matthewzeiler.com/mattzeiler/deconvolutionalnetworks.pdf>
- EmilienDupont, *wgan-gp* GitHub repository
Consulted for reference while implementing WGAN-GP ideas; code was not copied or directly reused.
<https://github.com/EmilienDupont/wgan-gp>